PROJECT WORK REPORT

on

DESIGN AND IMPLEMENTATION OF FIR FILTER USING POSIT ARITHMETIC FOR EEG SIGNAL PROCESSING

Work carried out at

BANGALORE INSTITUTE OF TECHNOLOGY

Submitted in partial fulfillment of the requirement for the award of

Bachelor of Engineering Degree

in

Electronics and Telecommunication Engineering

of

Visvesvaraya Technological University, Belagavi

by

K Raghavendra Rao	USN: 1BI21ET014
Karthik G K	USN: 1BI21ET015
Nandan K	USN: 1BI21ET029
Akarsh Dhiraj Shetty	USN: 1BI21ET002

Under the guidance of

Prof. N Chetan Anand

Associate Professor

Dept. of Electronics and Telecommunication Engineering

B.I.T., Bengaluru

DEPARTMENT OF ELECTRONICS AND TELECOMMUNICATION ENGINEERING

BANGALORE INSTITUTE OF TECHNOLOGY, BENGALURU – 560004

2024 – 2025

BANGALORE INSTITUTE OF TECHNOLOGY

K. R. Road, V. V. Pura, Bengaluru-560004

Department of Electronics and Telecommunication Engineering

CERTIFICATE

Certified that the Project work entitled **"DESIGN AND IMPLEMENTATION OF FIR FILTER USING POSIT ARITHMETIC FOR EEG SIGNAL PROCESSING"** work carried out by K Raghavendra Rao **(1BI21ET014),** Karthik G K **(1BI21ET015),** Nandan K **(1BI21ET029),** Akarsh Dhiraj Shetty **(1BI21ET002),** bona fide students of **Bangalore Institute of Technology** in partial fulfillment for the award of **Bachelor of Engineering** Degree in Electronics and Telecommunication Engineering of the Visvesvaraya Technological University, Belagavi during the year 2024-25. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the Departmental Library. The Project Report has been approved as it satisfies the academic requirements in respect of Project Work prescribed for the said Degree.

Guide	**HOD**	**Principal**
Prof. N Chetan Anand	**Dr. M Rajeswari**	**Dr. Aswath M.U**

External Viva:

Name of the Examiners Signature with date

1.

2.

ACKNOWLEDGEMENT

It would be our privilege to express our heartfelt gratitude and respect to all those who guided us in the completion of this Project work. We are highly indebted to **Rajya Vokkaligara Sangha** trust for introducing a great Educational Institute for studies.

We express our deep sense of gratitude to our Principal, **Dr. Aswath M.U.**, for providing an excellent academic environment which enabled us to complete the Project work.

We would like to express our heartfelt gratitude to our Head of Department, Department of Electronics and Telecommunication Engineering**, Dr. M Rajeswari.**, for her constant support during the tenure of the Project work and throughout our course of Engineering in the Institution.

We would like to express our heartfelt gratitude to our Project coordinator **Prof. Sudha. B**, Assistant Professor , Department of Electronics and Telecommunication Engineering**,** for her constant support and guidance during the course of the Project Work.

We would like to express our heartfelt gratitude to our Project Internal Guide, **Prof. N Chetan Anand,** Associate Professor, Department of Electronics and Telecommunication Engineering, for her constant support and guidance during the course of the Project work.

We are grateful to all the Teaching and Non-Teaching Staff, Department of Electronics and Telecommunication Engineering**,** who have been an inspirational support.

Last but not the least; we would like to thank our parents and friends, for their constant help and moral support throughout the completion of the Project work.

ABSTRACT

This project focuses on the design and implementation of a Finite Impulse Response (FIR) filter using posit arithmetic for Electroencephalogram (EEG) signal processing. Posit arithmetic, known for its enhanced precision and dynamic range over traditional floating-point systems, is leveraged to improve the accuracy and efficiency of EEG signal filtering. The FIR filter is designed to meet the specific requirements of EEG signal processing, including noise reduction and the preservation of critical signal features within the desired frequency band.

The filter's core components, such as the multiplier and adder, are implemented in Verilog HDL, ensuring efficient hardware realization. The design is validated through a comprehensive testing framework, including a Verilog-based testbench for functional verification. MATLAB is used to generate reference data by simulating the theoretical behavior of the FIR filter on EEG signals, facilitating a detailed comparison with the Verilog implementation.

This approach demonstrates the effectiveness of posit arithmetic in enhancing numerical precision for biomedical signal processing applications. The project provides a robust framework for designing and implementing efficient FIR filters tailored for EEG signal analysis, with potential applications in brain-computer interfaces, and neurological research.

TABLE OF CONTENTS

SL. NO	TOPIC	Page NO
	Acknowledgment	
	Abstract	**i**
	Table of contents	**ii**
	List of Figures	**iv**
	List of Tables	**v**
	List of Abbreviations	**vi**
1	**Introduction**	**1**
2	**Literature Survey**	**2**
3	**IEEE Standards 754 Representation**	**4**
	3.1 Evolution	4
	3.2 IEEE Standard 754 Representation	5
	3.3 Exceptions of IEEE 754 Standard	6
	3.4 Issue with IEEE Floats	7
4	**Posit Arithmetic**	**8**
	4.1 Background: Type I and Type II Unums	8
	4.1.1 TypeIUnums	8
	4.1.2 TypeII Unums	8
	4.2 The Posit Format	9
	4.2.1 Understanding a N-bit Posit	10
	4.2.2 Exceptions of Posit	10
5	**Operations ON Posits**	**11**
	5.1 Addition	11
	5.2 Multiplication	13
6	**FIR Filter Structure**	**15**
7	**Conversion**	**17**
	7.1 Binary to Posit	17
	7.2 Decimal to Posit	17
	7.3 Posit to Binary	18
8	**Result and conclusion**	**19**
	8.1 Adder Output	20

8.2 Multiplier Output 21
8.3 FIR Filters outputs 21
8.4 Conclusion 25
Appendix

LIST OF FIGURES

Figure No	Title	Page No
4.1	Type I unum representation	8
4.2	Projective real number line mapped to 4-bit two's complement integers	9
4.3	Posit format for finite non-zero values	9
5.1	Steps involved in implementing addition operation	6
5.1	Steps involved in implementing multiplication operation	10
8.1	Output for Adder	19
8.2	Output for Multiplier	20
8.3	FIR input signal	22
8.4	FFT plot of output	22
8.5	FIR output signal	23
8.6	30Hz input signal (sine wave)	23
8.7	Out of band FIR input signal	24
8.8	FFT of FIR output signal	25

LIST OF TABLES

Table No	Name of the Particulars	Page No
3.1	Characteristic parameters of IEEE-754 formats	5
4.1	Numerical meaning k of regime bits	10
4.2	Useed as a function of es	10
8.1	Stop band attenuation	25

LIST OF ABBREVIATIONS

Abbreviation	Description
EEG	Electroencephalogram
FIR	Finite Impulse Response
UNUMS	Universal Numbers
LOD	Leading One Detector
NAN	Not-A-Number

Chapter 1

INTRODUCTION

Electroencephalogram (EEG) signal processing plays a vital role in biomedical engineering, enabling critical applications such as brain-computer interfaces, seizure detection, and neurological disorder analysis. However, EEG signals are often contaminated with noise and artifacts, necessitating precise filtering to isolate meaningful information within specific frequency bands. The Finite Impulse Response (FIR) filter is widely used for this purpose due to its stability and ability to achieve a linear phase response, ensuring minimal distortion of the signal's essential features.

In this project, posit arithmetic is introduced as an innovative approach to FIR filter design. Posit arithmetic offers a superior alternative to traditional floating-point systems, providing higher precision, dynamic range, and energy efficiency in hardware implementations. These properties are particularly advantageous for processing EEG signals, which require a high degree of numerical accuracy to maintain signal fidelity.

The design and implementation of the FIR filter are carried out using Verilog HDL, focusing on optimizing the filter’s arithmetic operations, including multiplication and addition, to accommodate posit representation. Verification of the design involves testing with synthetic and real EEG signals. A Verilog-based testbench simulates the filter’s functionality, while MATLAB serves as a reference tool to validate the hardware results by providing benchmark data based on theoretical simulations.

This project aims to combine advanced numerical techniques and efficient hardware design to deliver a robust filtering solution for EEG signal processing. The successful implementation of this methodology highlights the potential of posit arithmetic in biomedical signal processing applications, paving the way for more accurate and resource-efficient systems in healthcare technology.

Chapter 2

LITERATURE SURVEY

	BOOK,PAPER REFRENCE AUTHOR	REFERED PARAMETER	DESCRIPTION
1.	Posit Arithmetic, John L. Gustafson	Encoding ,Decoding and posit Representation .	In posit mode, a unum behaves much like a floating-point number of fixed size, rounding to the nearest expressible value if the result of a calculation is not expressible exactly; however, the posit representation offers more accuracy and a larger dynamic range than floats with the same number of bits, as well as many other advantages.
2.	Hardware Optimization and Implementation of a 16-Channel Neural Tree Classifier for on-chip Closed-Loop Neuromodulation, A. P. Sharma, K. A. Rao and L. Somappa	Filter Structure, Window	Using Transposed Filter Structure for reducing hardware components, . Using Kaiser 5.65b windowing for stopband attenuation of -60dB(Ideal case).

3.	An Investigation on Inherent Robustness of Posit Arithmetic	Posit bit size (N) Exponent size (es) Error Rate Computation Precision Power Consumption	This paper investigates the robustness of the Posit arithmetic system in contrast to traditional floating-point systems like IEEE 754. The focus is on the accuracy, precision, and overall efficiency when using Posits in various computational scenarios, particularly under extreme values and rounding errors. Through simulations, the authors compare performance and error margins between Posit and floating-point arithmetic, emphasizing its advantages in high-precision scientific computing tasks.

Chapter 3
IEEE STANDARD 754 FLOATS

3.1 Evolution

The world of numerical computation changed much in 1985, when the IEEE 754- 1985 Standard for Binary Floating-Point Arithmetic was released [1]. This standard specifies various formats, the behavior of the basic operations and conversions, and exceptional conditions. As a matter of fact, the Intel 8087 mathematic co-processor, built a few years before, in 1980, to be paired with the Intel 8088 and 8086 processors, was already extremely close to what would later become the IEEE 754-1985 standard. Now, most systems of commercial significance offer compatibility with IEEE 754-1985. This has resulted in significant improvements in terms of accuracy, reliability, and portability of numerical software. William Kahan played a leading role in the conception of the IEEE 754-1985 standard and in the development of smart algorithms for floating-point arithmetic.

IEEE 754-1985 and 854-1987 have been under revision since 2001. The new revised standard, called IEEE 754-2008 in this book, merges the two old standards and brings significant improvements. It was adopted in June 2008 [2].

The standard defines:

• Arithmetic formats: sets of binary and decimal floating-point data, which consist of finite numbers (including signed zeros and subnormal numbers), infinities, and special ”not a number” values (NaNs)

• Interchange formats: encodings (bit strings) that may be used to exchange floating-point data in an efficient and compact form

• Rounding rules: properties to be satisfied when rounding numbers during arithmetic and conversions

• Operations: arithmetic and other operations (such as trigonometric functions) on arithmetic formats

• Exception handling: indications of exceptional conditions (such as division by zero, overflow, etc.)

3.2 IEEE Standard 754 Representation

An integer can be represented as a binary string of certain amount of bits which is a combination of three fields: sign, exponent and fraction as below:

S	Exponent	Fraction

$$(X)_{10} = (-1)^S \times 2^{(\text{Exponent}-\text{Bias})} \times (1.\text{Fraction})_2$$

This table represents the characteristics parameter of IEEE-754 formats. The MSB of the bit string is the sign(S) of the number. If MSB is 0, the number is positive and if it is 1 the number is negative. The n-bit exponent field is an unsigned integer. The exponent field should be able to represent both positive and negative exponent value. To achieve this we add a bias of 2n−1 − 1 to the actual exponent while representing as IEEE float. When it comes to their precision and width in bits, the standard defines four basic formats: Half precision, Single precision, Double precision and Quadruple precision. The Bias coefficient is equal to 127 for an 8bit exponent of a single precision format and 1023 for 11 bit exponent of 64 bit double precision format. This allows usage of exponents in the range of -127 to +128 which corresponds to 0 to 255 for an 8 bit exponent of single precision format and -1023 to +1024 corresponding to 0 to 2047 for a 11bit exponent of double precision format. The single-precision format offers a range from 2^{-127} to 2^{127}, which is equivalent to 10^{-38} to 10^{38} and it ranges from $2^{-1023}$to 2^{1023}, which is equivalent to the range of 10^{-308} to 10^{308},in case of double precision format.[3]

Table 3.1: Characteristic parameters of IEEE-754 formats

Precision	No.of Bits	Exponent Bits	Fraction Bits	Exponent Bias
Half Precision	16	5	10	15
Single Precision	32	8	23	127
Double Precision	64	11	52	1023
Quadruple Precision	128	15	112	16383

3.3 Exceptions of IEEE 754 standard

The standard reserves all 0s and all 1s of the exponent field to handle some exceptional cases:

• **Zero:** As mentioned above, zero is not directly representable in the straight format, due to the assumption of a leading 1 (we'd need to specify a true zero mantissa to yield a value of zero). Zero is a special value denoted with an exponent field of zero and a fraction field of zero. Because of the sign bit there is a distinct representations for -0 and +0, though they both compare as equal.

• **Denormalized:** Numbers If the exponent have all 0s, but the fraction is nonzero (else it would be interpreted as zero), then the value is a denormalized number, which does not have an assumed leading 1 before the binary point. Thus, this represents a number (−1)^s × 0. f × 2^ −126 , where s is the sign bit and f is the fraction. For double precision, denormalized numbers are of the form (−1)^s × 0. f × 2^ −1022 . From this you can interpret zero as a special type of denormalized number.

• **Infinity:** The values +infinity and -infinity are denoted with an exponent of all 1s and a fraction of all 0s. The sign bit distinguishes between negative infinity and positive infinity. Being able to denote infinity as a specific value is useful because it allows operations to continue past overflow situations. Operations with infinite values are well defined in IEEE floating point.

• **NaN** The value NaN (Not a Number) is used to represent a value that does not represent a real number. NaN's are represented by a bit pattern with an exponent of all 1s and a non-zero fraction. There are two categories of NaN: QNaN (Quiet NaN) and SNaN (Signalling NaN). They have the following format, where s is the sign bit:

– QNaN: s 11111111 10000000000000000000000

– SNaN: s 11111111 00000000000000000000001

A QNaN is a NaN with the most significant fraction bit set. QNaN's propagate freely through most arithmetic operations. These values pop out of an operation when the result is not mathematically defined. An SNaN is a NaN with the most significant fraction bit clear. It is used to signal an exception when used in operations. SNaN's can be handy to assign to uninitialized variables to trap premature usage. Semantically, QNaN's denote indeterminate operations, while SNaN's denote invalid operations.

3.4 Issues With IEEE Floats

• **Wasted Bit Patterns -** 32-bit IEEE floating point has around sixteen million ways to represent Not-A-Number (NaN) while 64-bit floating point has nine quadrillion. A NaN is an exception value for invalid operations such as division by zero.

• **Mathematically Incorrect** - The format specifies two zeroes, a negative and positive zero which have different behaviors.

• **Overflows to ±inf and underflows to 0** - Overflowing to ±inf increases the relative error by an infinite factor, while underflowing to 0 loses sign information.

• **Complicated Circuitry** - One of the reasons why IEEE floating points have complicated circuitry is because the standard defines support for denormalized numbers. Denormalized floating point numbers have a hidden bit of 0 instead of 1.

• **No Gradual Overflow and Fixed Accuracy** - If accuracy is defined as the number of significand bits, IEEE floating point have fixed accuracy for all numbers except denormalized numbers because the number of significant digits is fixed. Denormalized numbers are characterized by a decreased number of significand digits when the value approaches zero as a result of having a zero hidden bit. Denormalized numbers fill the underflow gap (i.e. the gap between zero and the least non-zero values). The counterpart for gradual underflow is gradual overflow which does not exist in IEEE floating point.

Chapter 4

POSIT ARITHMETICS

4.1 Background: Type I and Type II Unums :

The unum (universal number) format is a format similar to floating point, proposed by John Gustafson as an alternative to the now ubiquitous IEEE 754 format[6].

4.1.1 Type-I Unums:

The Figure 4.1 shows ”Type I” unum is a superset of IEEE 754 Standard floating-point format[4]. It uses a ”ubit”at the end of the fraction to indicate whether a real number is an exact float or lies in the open interval between adjacent floats. While the sign, exponent, and fraction bit fields take their definition from IEEE 754, the exponent and fraction field lengths vary automatically, from a single bit up to some maximum set by the user. Type I unums provide a compact way to express interval arithmetic, but their variable length demands extra management. They can duplicate IEEE float behavior, via an explicit rounding function.

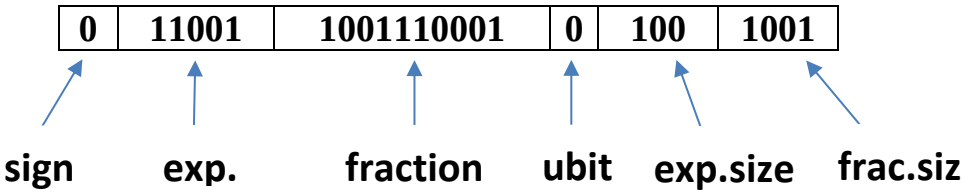

Figure 4.1: Type I unum representation

4.1.2 Type II Unums :

The Figure 4.2 shows Projective real number line mapped to 4-bit two’s complement integers. ”Type II” unum [5] abandons compatibility with IEEE floats, permitting a clean, mathematical design based on the projective reals. The key observation is that signed (two’s complement) integers map elegantly to the projective reals, with the same wraparound of positive numbers to negative numbers, and the same ordering. Like Type I, Type II unums ending 1(ubit) represent the open interval between adjacent exact points (unums ending in 0). The structure of 5-bit Type II unums is shown in fig3.2. For an n-bit Type II unum, the upper right quadrant corresponds to an ordered set of $2n-3-1$ real numbers xi . The upper left quadrant represents the negative values of xi , the reflection about the vertical axis represents the opposite numbers

of the upper right quadrant. The lower semi circle, the reflection about the horizontal axis represents the reciprocals of the numbers in the upper half circle. This horizontal projection makes ×and÷ symmetrical operation like + and -.

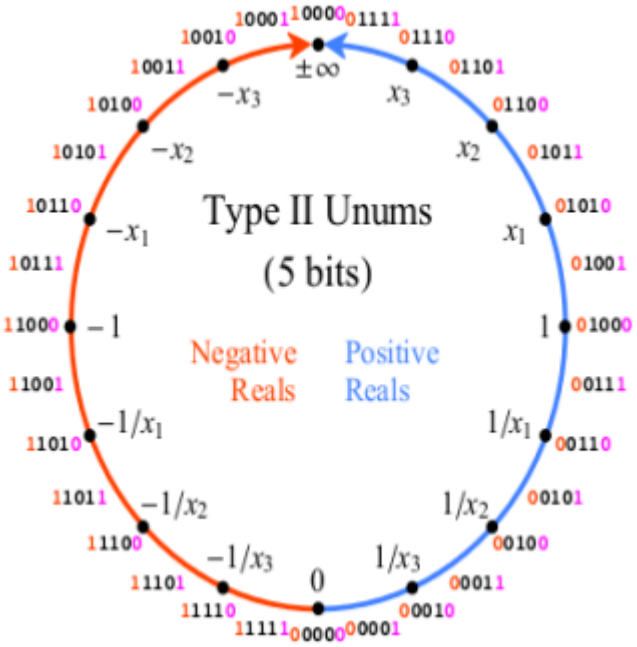

Figure 4.2: Projective real number line mapped to 4-bit two's complement integers

4.2 The Posit Format

The below Figure 4.3 represents the structure of an n-bit posit with es exponent bits. The n-bits of posit consists of four fields sign, regime, exponent and fraction.

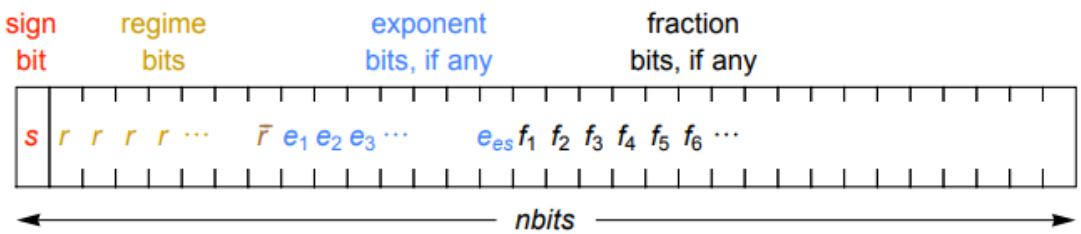

Figure 4.3: Posit format for finite non-zero values

4.2.1 Understanding a n-bit posit

- **Sign** - The MSB of the n-bits represents the sign of the number represented. If it is 0, the number is positive and if 1 the number is negative. If sign bit is 1, take 2's complement of the entire posit before decoding it.

- **Regime bits** : The Table 4.1 shows Numerical meaning k of regime bits. The identical bits after the sign bit either terminated by opposite bit or the end of the binary string is reached contribute to the regime bits. Let there be 'm' identical bits in the regime. If the m bits are 0, then run length(k)= -m; if they are 1s, then k=m-1. Table below gives more clarification about regime bits and numerical meaning of run length(k).

Table 4.1: Numerical meaning k of regime bits

Binary	0001	001x	01xx	10xx	110x	1110
Run-length,k	-3	-2	-1	0	1	2

The regime contributes to a scaling factor of useed k , where useed = $2^{2^{es}}$

Table 4.2: useed as a function of es

Es	0	1	2	3	4
useed	2	2^2=4	4^2=16	16^2=256	256^2=65536

• **Exponent Bits** - The Table 4.2 shows useed as a function of es. Bits after regime bits are exponent bits. There can be up to es exponent bits. These are regarded as an unsigned integer. They represent a scaling factor of 2e . There is no bias as there is for floats

• **Fraction bits** - The bits left after the exponent bits represent the fraction, just like the fraction 1. f in a float, with a hidden bit 1. Posits doesn't support the de-normalized format with hidden bit 0.

3.2.2 Exceptions of Posits

There only two exceptions for posits.

1. Zero : All n-bits are 0s.

2. ± Infinity : All the n-bits except the MSB are 0s. (1 followed by all 0 bits).

The decimal equivalent of a posit is calculated as below:

(X)10 = (−1)^S × (used)^k × 2^exp × (1.Fraction)^2

Chapter 5

OPERATIONS ON POSITS

Once the standard has been explained it is time to start with the implementation of the arithmetic operations using the posit representation. In this chapter we deal with different arithmetic operations such as Addition, Multiplication and their corresponding algorithms [8].

5.1 Addition:

Addition is the fastest of the fundamental arithmetic operations, but also surprisingly complex. The change from fixed to floating-point turns the simple addition into a 10 step process:

1. Convert to internal representation
2. Find exponent difference
3. Allign significands
4. Add significands
5. Round
6. Detect leading one
7. Normalize result
8. Adjust exponent
9. Convert to Posit format

The Figure 5.1 shows the flow chart for adder ,First the input posits are decoded and exponents and mantissa bits are separated. The exponents are compared and the the absolute difference between them is found. The mantissa with the lower exponent is aligned so that they have same exponents. The result exponent will be max(Exp1, Exp2). Then the two mantissas are added or subtracted depending on the effective addition/subtraction operation.

After the addition/subtraction the significand may not fit in the format anymore as it may be a digit too large, or it may be unnormalized. A Leading One Detector (LOD) finds the position of first non-zero digit and a shifter normalize the result to a format obeying the input format. The exponent is updated with the shift. After normalization inexact results must be rounded according to the current rounding modes. As the rounding may overflow the significand has to be normalized again and the exponent must be updated accordingly. Finally, the internal format must be converted to posit format.

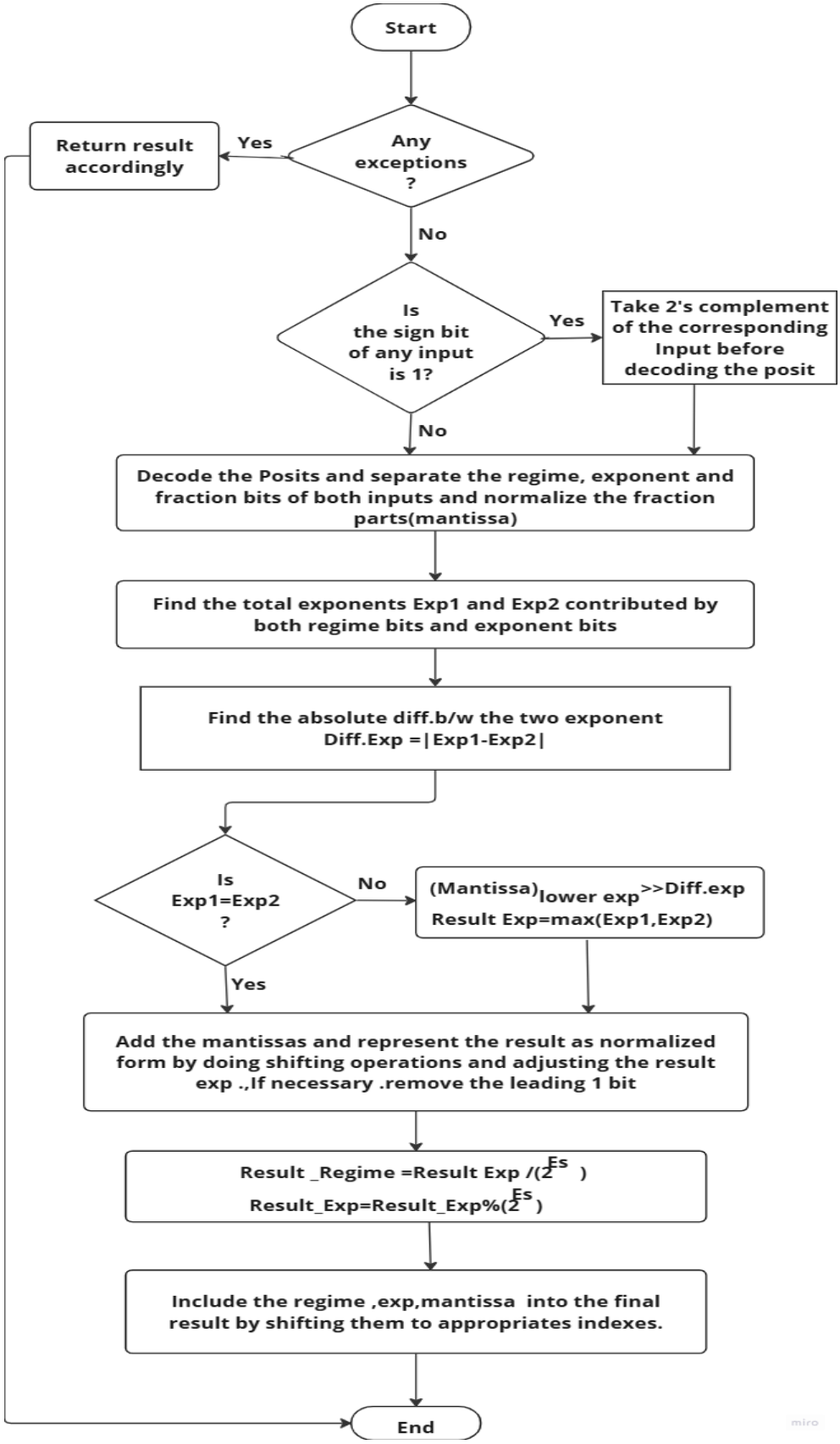

Figure 5.1: Steps involved in implementing addition operation

5.2 Multiplication

Floating-point multiplication is much simpler in concept compared to addition, but it is also slower. In total it is a 7-step process where step 2 to 4 conceptually differ from addition

1. Convert to internal representation

2. Multiply significands and add exponents in parallel

3. Detect leading one

4. Normalize result

5. Round

6. Adjust exponent

7. Convert to posit format

The Figure 5.2 shows flow chart for multiplying, After decoding the input posits, the exponents are added and is assigned as the result exponent. The product of the mantissas is found and then leading zero is detected from the product for normalizing. The product is normalized by shifting and adjusting the result mantissa. Rounding the result of the mantissas product is done if necessary, according to the Round rules followed. The regime bits and the exponent bits are deducted from the result exponent. The leading 1 one from the mantissa's product is removed before converting into posit format.

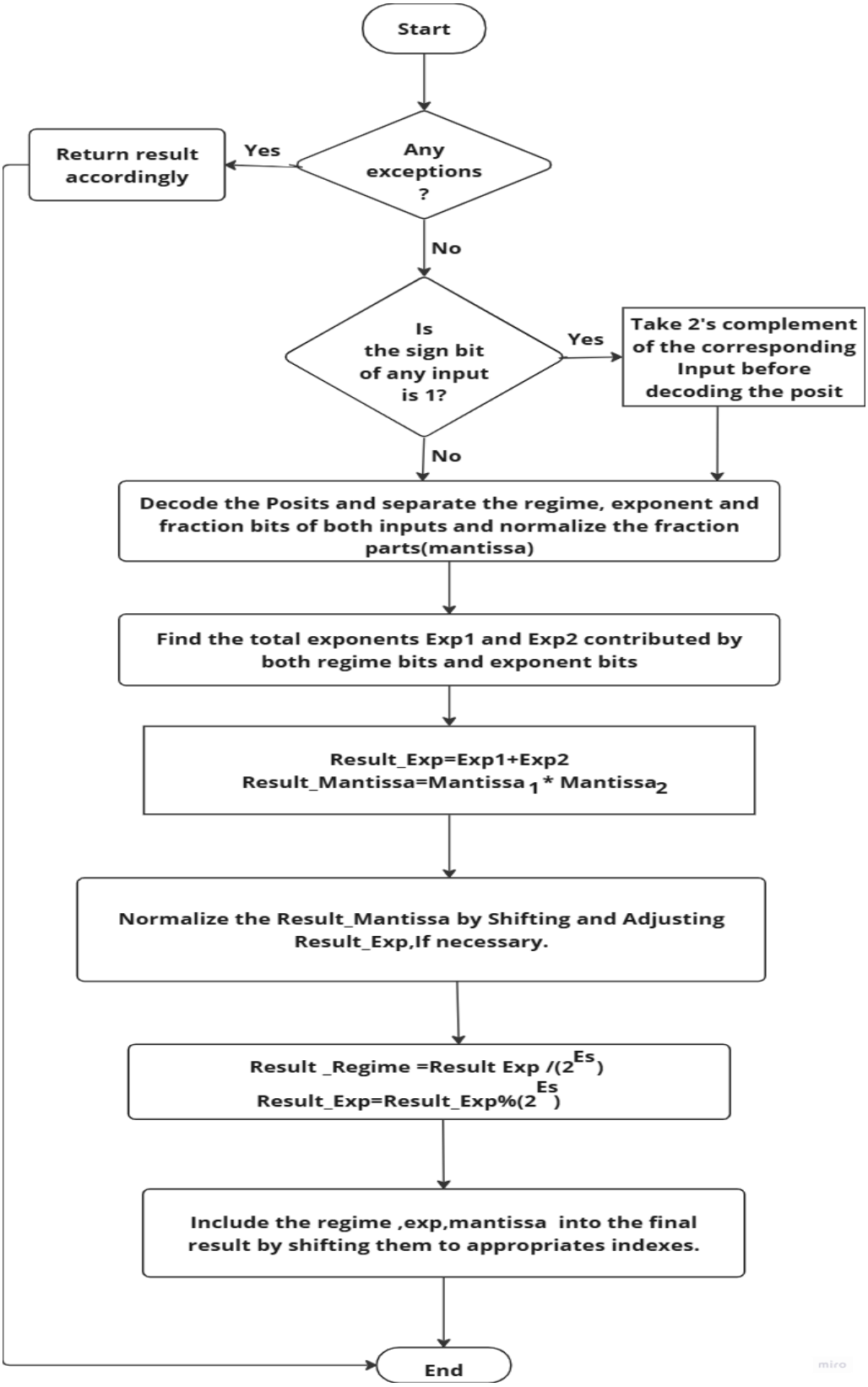

Figure 5.2: Steps involved in implementing multiplication operation

Chapter 6

FIR FILTER STRUCTURE

The Figure 6.1 illustrates two structures of Finite Impulse Response (FIR) filters: the Regular Transposed FIR filter structure and the Linear Phase FIR filter structure with symmetric coefficients [7]

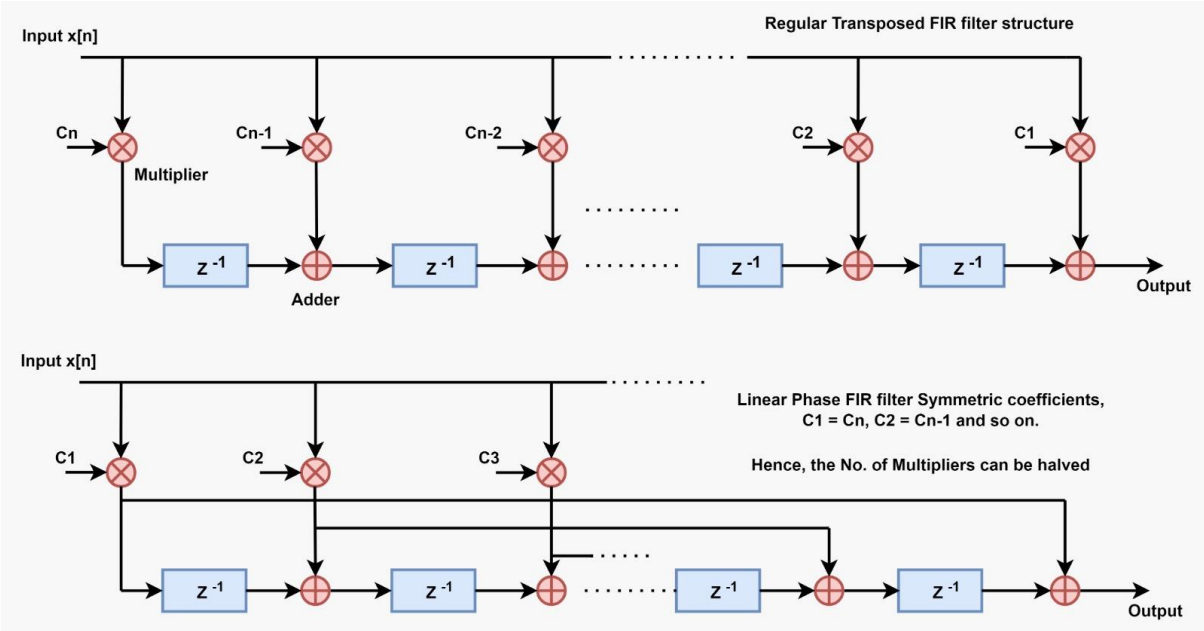

Figure 6.1 Transposed FIR filter structure

- **Regular Transposed FIR Filter Structure:** The Regular Transposed FIR Filter Structure is a widely used architecture for implementing Finite Impulse Response (FIR) filters. It consists of a series of multipliers, adders, and delay elements. The input signal x[n] is fed into the structure, where it passes through delay elements (z^−1) that represent unit delays. At each stage, the delayed signal is multiplied by a corresponding filter coefficient (Cn,Cn−1,Cn−2,…) These multipliers compute the weighted contributions of the signal components, and the results are summed using adders to produce the final output. This structure is highly modular and allows straightforward implementation of the FIR filtering equation, making it suitable for general-purpose applications. However, it can be computationally expensive in terms of hardware, as it requires one multiplier per coefficient. This structure is particularly favored for its simplicity and versatility in realizing arbitrary FIR filters.

- **Linear Phase FIR Filter with Symmetric Coefficients:** The Linear Phase FIR Filter with Symmetric Coefficients leverages the inherent symmetry of coefficients in linear-phase FIR filters to optimize hardware and computational efficiency. In such filters, the coefficients exhibit symmetry llowing the pairing of input samples that are equidistant from the center of the filter. These paired samples are first summed and then multiplied by the corresponding shared coefficient, effectively halving the number of multiplications required compared to the regular FIR structure. This optimization reduces hardware complexity and power consumption while maintaining the filter's performance and response. The symmetric design is particularly advantageous in applications requiring efficient and cost-effective FIR filter implementations.

Chapter 7

CONVERSIONS

7.1 Binary to Posit

The Python script converts binary strings into the Posit number format, a highly efficient floating-point representation balancing range and precision. It begins by checking if the input binary string is all zeros, directly returning it since further processing is unnecessary. For signed binary numbers, the code computes the two's complement, achieved by inverting all bits and adding one to the result.

Next, the fractional part of the normalized value is used to calculate the mantissa, represented as a binary string of a specified length. The Posit number is then assembled by concatenating its components: the sign bit (indicating positive or negative), regime bits (encoding the scaling factor using a specific pattern), exponent bits (providing additional precision within the regime’s range), and mantissa bits (representing the fractional precision). This process is repeated for each binary string read from an input file, with the resulting Posit numbers written to an output file. The script demonstrates the efficient encoding of numbers using the Posit format, providing better accuracy and dynamic range compared to traditional floating-point representations.

7.2 Decimal to Posit

The Python script converts decimal numbers into the Posit format, a numerical representation optimized for balancing precision and dynamic range in floating-point arithmetic. When the input decimal value is zero, the script directly returns a binary string of zeros padded to match the specified bit width, as no further processing is required. For non-zero values, the script begins by scaling the absolute value of the input using a useed factor.

After scaling, the number is normalized into the range [1,2][1, 2][1,2] by iteratively dividing it by 2, incrementing the exponent to reflect the number of divisions. The fractional part of the normalized value is then extracted and converted into a binary mantissa, which represents the precision of the Posit number. These components—sign bit, regime, exponent, and mantissa—are combined to construct the final Posit number. The sign bit indicates whether the number is positive or negative, the regime bits encode the scaling factor based on k, the exponent bits

provide additional scaling within the regime, and the mantissa bits ensure precision. The resulting Posit number is truncated to the desired bit width for consistency.

For negative inputs, the final Posit binary representation undergoes a two's complement operation to encode the sign properly. The script reads decimal numbers from an input file, processes each number into its corresponding Posit representation, and writes the results to an output file. This process demonstrates the efficiency and adaptability of the Posit format in encoding a wide range of numbers with high precision.

7.3 Posit to Binary

The regime value kk is extracted from the regime bits, which indicate how many times the number has been scaled by a factor of useed .The regime bits are analyzed to determine the number of scaling steps by checking the pattern of consecutive ones or zeros. After determining kk, the script extracts the exponent bits, which are then converted from binary to decimal and used to adjust the scaling of the number.

The mantissa part of the Posit number is then processed, where the binary mantissa is converted back to a fractional value. The fractional part is added to 1, as Posit uses a hidden leading bit, giving the final mantissa value. The script then combines these components: the sign bit, the regime value kk, the exponent, and the mantissa, and calculates the final decimal value by applying the appropriate scaling and exponentiation.

The resulting decimal value is then returned. The script also processes multiple Posit strings from an input file, performs the conversion for each, and writes the corresponding decimal values to an output file. This approach efficiently handles the Posit format, enabling accurate conversion between Posit and decimal representations while maintaining precision and supporting both positive and negative numbers.

Chapter 8

OUTPUTS

8.1 Adder output:

The Figure 8.1 shows output for adder which has two inputs**,** if sign bit is 1the value is converted into absolute value. Decoding the posits and separate the regime, exponent and fraction bit of both inputs and normalize the fraction part. Finding total exponents Exp1 and Exp2, contributed by both regime bits and exponents bits. Finding the absolute difference between the two exponents. Adding the fraction and representing the result as a normalized form by doing shifting operations. Once we get the regime bit and exponent bit we can write the output for adder.

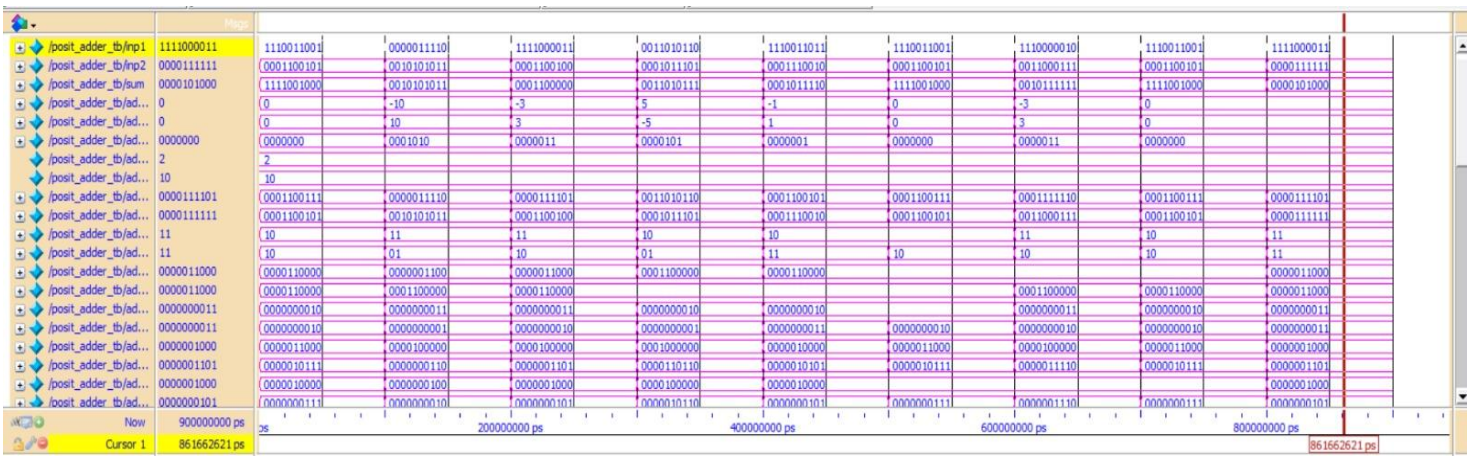

Figure 8.1 Output for Adder

Manual Calculations for Adder**:**

The manual Calculations for Adder, where theoretical and manually calculated values are matching.

Inp1=1111000011 ⟶ 0000111100 ⟶ 0000111101 ⟶ 2's complement

Inp2=0000111111

0000111101

0000111111

K1=-3 Exp1=3 Fraction1=1.101

K2=-3 Exp2=3 Fraction2=1.111

T_Exp1=-3*2^{es}+3=-12+3=-9

T_Exp2=-3*2^{es}+3=-12+3=-9

Result_Exp=-9 -2 = -11

Result_fraction = Fraction1- Fraction2 =|1.101 – 1.111|=0.01 (shift bits to left because fraction should not be less than 1 and greater than 2)

Result_Fraction =1.00

Result_Regime = $\frac{\text{Result_Exp}}{2^{es}} = \frac{-11}{4}$ Q = -2-1 K= -3

Result_Regime = 3*4= (12-11)=1

So Exp =1

final ouput = 0000101000

8.2 Multiplier Output:

The Figure 8.2 shows output for multiplier which has two inputs**,** if sign bit is 1the value is converted into absolute value. Decoding the posits and separate the regime, exponent and fraction bit of both inputs and normalize the fraction part. Finding total exponents Exp1 and Exp2, contributed by both regime bits and exponent's bits. Finding the absolute difference between the two exponents. Multiplying the fraction and representing the result as a normalized form by doing shifting operations. Once we get the regime bit and exponent bit we can write the output for multiplier.

Fig 8.2 Output for Multiplier

Manual Calculation for Multiplier**:**

Inp1=0011101101

Inp2=1111111001 ⟶ 0000000110 ⟶ 0000000111 ⟶ 2's complement

K1=-1 Exp1=3 Fraction1=1.01101

K2=-6 Exp2=3 Fraction2=1.00000

T_Exp1=-1*2^{es}+3=-4+3=-1

T_Exp2=-6*2^{es}+3=-24+3=-21

Result_Exp=T_ Exp1 + T_ Exp2 = -1-21 = -22

Result_fraction = Fraction1* Fraction2

= 1.01101*1.00000

=1.01101

Result_Regime = $\frac{\text{Result_Exp}}{2^{es}} = \frac{-22}{4}$ Q = -5-1 K= -6

-6*4= -24

|-24-(-22)|= 2

So Exp=2

0000000110 ⟶ take 2's complemen to get final ouput ,because the inp2's sign bit is negative so final ouput = 1111111010

8.3 FIR Filters outputs:

The Figure 8.3 below shows 10 bit quantized input signal given at the fir input of frequency 10 Hz, which is the in-band signal for a band pass filter having lower cutoff frequency of 8 Hz and upper cutoff frequency of 13 Hz. This band is also called alpha band in EEG signal processing.

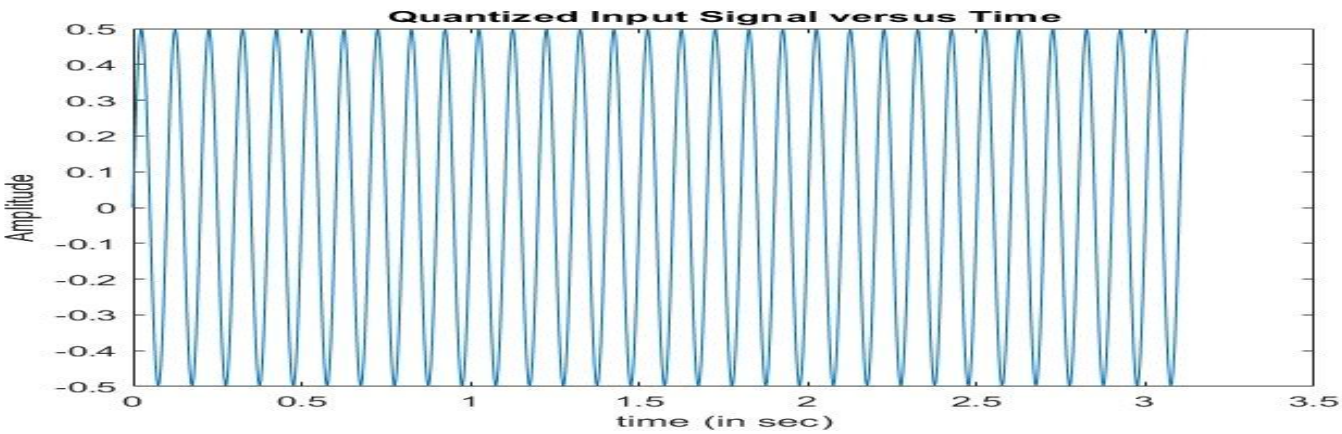

Figure 8.3 FIR Input Signal

Figure 8.4 and 8.5 shows the FFT plot of the FIR output and the output of fir filter respectively. Taking it's FFT, we see that the output signal frequency is 10 Hz, which matches with a frequency of the input, Hence this proves that the filter is performing according to it's specifications.

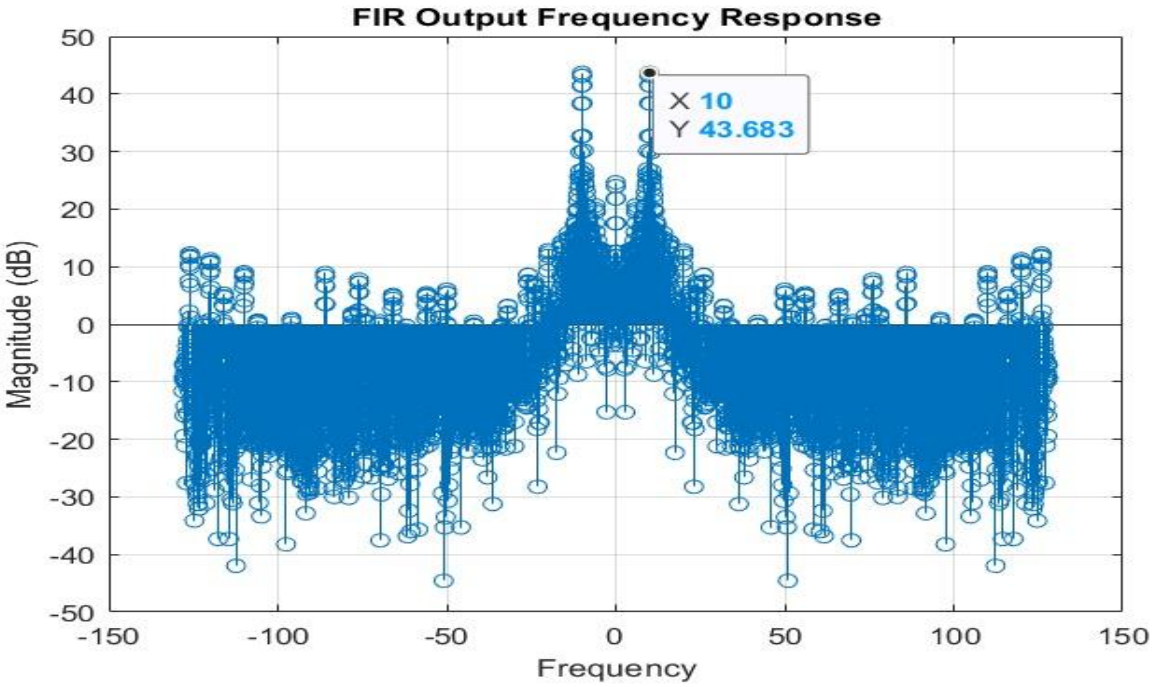

Figure 8.4 FFT Plot of output

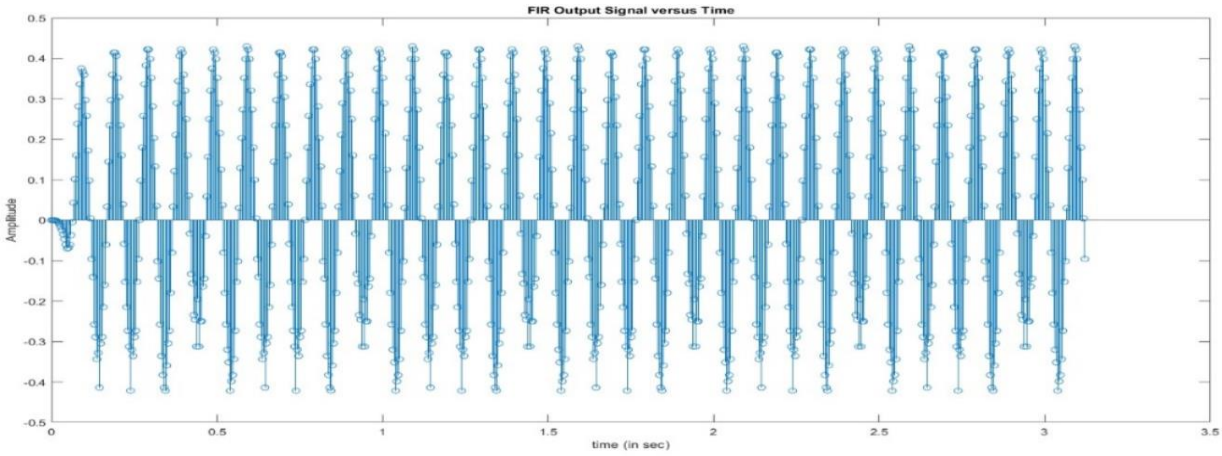

Figure 8.5 FIR Output Signal

Figure 8.6 is the out of band input sine wave of frequency 30Hz.The signal is given to the input for making sure that the filter attenuates any signal that is outside the range of 8-13Hz.

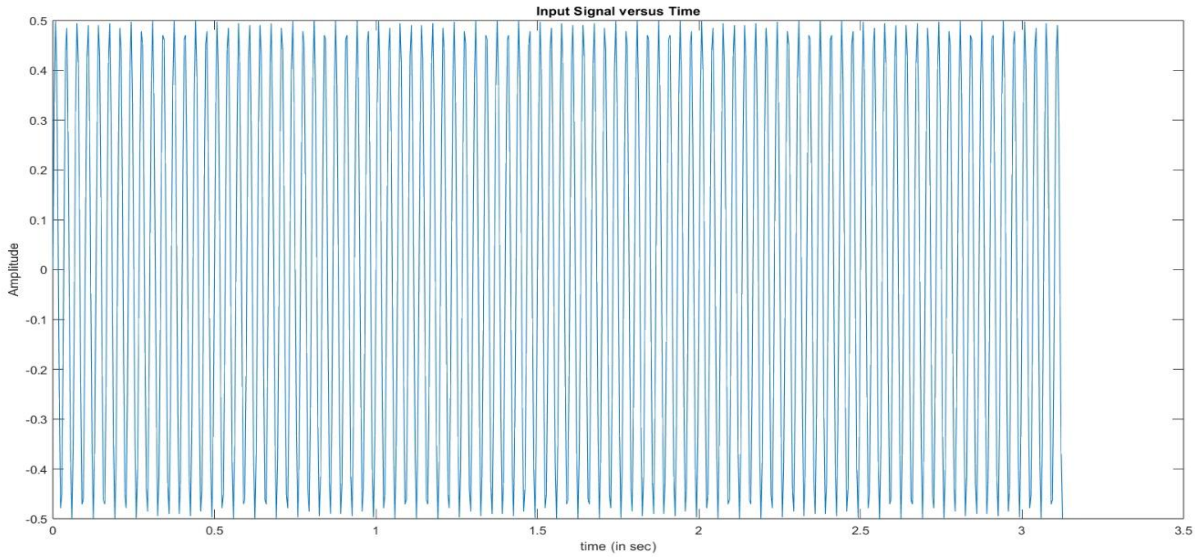

Figure 8.6 30Hz Input Signal (Sine Wave)

Fig 8.7 and 8.8 shows the fir output signal and its FFT plot for the input signal of frequency 30Hz. From fig 8.7, it is evident that the input signal is attenuated after being passed through the FIR filter. This shows that the filter performs proper attenuation for the given specifications. From fig 8.8 the attenuation of other frequencies is evident.

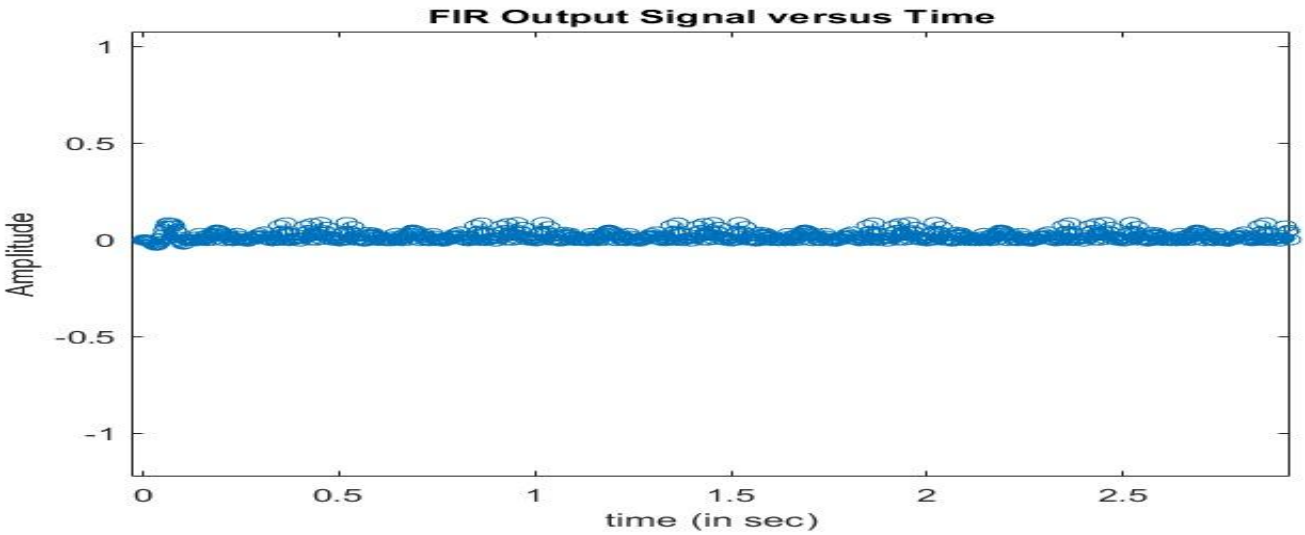

Figure 8.7 Out of Band FIR Input Signal

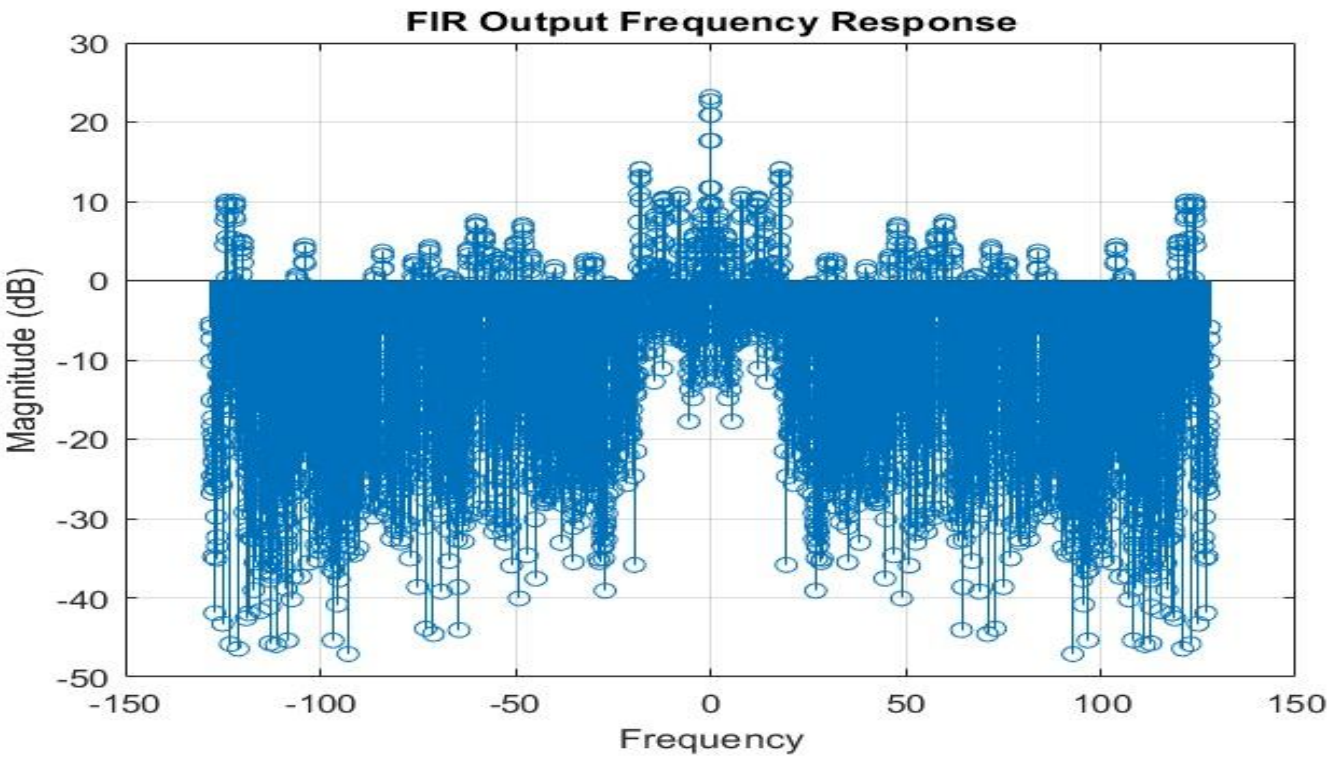

Figure8.8 FFT of FIR Output Signal

8.4 Conclusion:

The table 8.1 compares the stop-band attenuation (in dB) of Posit and Fixed Point representations across five EEG frequency bands: Gamma (30-50 Hz), Beta (13-30 Hz), Alpha (8-13 Hz), Theta (4-8 Hz), and Delta (1-4 Hz). Posit representation (-47.07 dB) demonstrates higher attenuation than Fixed Point (-44.77 dB) in the Gamma band. In the Beta band, Posit representation (-48.26 dB) significantly outperforms Fixed Point (-43.55 dB) in attenuation. Similarly, in the Alpha band, Posit representation (-46.60 dB) achieves better attenuation compared to Fixed Point (-41.78 dB). For the Theta band, both representations perform similarly, with Posit (-43.94 dB) having a slight edge over Fixed Point (-43.83 dB). In the Delta band, Posit representation (-46.49 dB) also achieves higher attenuation compared to Fixed Point (-42.8 dB). Overall, the Posit representation consistently achieves higher stop-band attenuation than the Fixed Point representation across all frequency bands, demonstrating its superiority for applications requiring effective signal filtering and noise suppression.

Table 8.1 Comparing Stop Band Attenuation Values Obtained by Kaiser Window for Posit and Fixed Point

Stop Band Attenuation (in dB)					
	Gamma (30-50) Hz	Beta (13-30) Hz	Alpha (8-13) Hz	Theta (4-8) Hz	Delta (1-4) Hz
Posit	-47.07	-48.26	-46.60	-43.94	-46.49
Fixed Point	-44.77	-43.55	-41.78	-43.83	-42.8

REFERENCES

[1] American National Standards Institute and Institute of Electrical and Eectronic Engineers. IEEE Standard for Binary Floating-Point Arithmetic. ANSI/IEEE Standard 754-1985, 1985.

[2] IEEE Computer Society. IEEE Standard for Floating-Point Arithmetic. IEEE Standard 754-2008, August 2008. available at http://ieeexplore.ieee.org/servlet/opac?punumber=4610933 .

[3] 754-2008 - IEEE Standard for Floating-Point Arithmetic. available at https://ieeexplore.ieee.org/servlet/opac?punumber=4610933

[4] Sharma, H. Singh, and R. Gupta, "Efficient Digital Filter for EEG Preprocessing," in Proceedings of the IEEE International Conference on Electronics, Circuits, and Systems (ICECS), Cairo, Egypt, 2020, pp. 87-90.

[5] H. Lee and J. Kim, "Hardware-Efficient FIR Filter for Signal Processing Applications," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 28, no. 4, pp. 702-711, Apr. 2020.

[6] John L Gustafson. Posit Arithmetic, John L.

[7] Hardware Optimization and Implementation of a 16- Channel Neural Tree Classifier for on-chip Closed-Loop Neuromodulation, A. P. Sharma, K. A. Rao and L. Somappa

[8] P.NAVEEN KUMAR. Posit Arithmetics.

APPENDIX

VERILOG CODE FOR ADDER:

```
module posit_adder #(parameter INP_SIZE = 10,
parameter ES = 2)
(
input [INP_SIZE-1:0] inp1,inp2,
output [INP_SIZE-1:0] sum
);
wire [INP_SIZE-1:0] abs_inp1, abs_inp2;
reg [3:0] k_val1, k_val2;
wire [2:0] regime_end_index1;
wire [2:0] regime_end_index2wire [ES-1:0] exp1;
wire [ES-1:0] exp2;
wire [INP_SIZE-1:0] exp_extractor1;
wire [INP_SIZE-1:0] exp_extractor2;
wire [INP_SIZE-1:0] exp_intermediate1; wire [INP_SIZE-1:0] exp_intermediate2;
wire [INP_SIZE-1:0] frac1;
wire [INP_SIZE-1:0] frac2;
wire [INP_SIZE-1:0] frac1_max_shifted_leftwire [INP_SIZE-1:0] frac2_max_shifted_left;
wire [INP_SIZE-1:0] frac1_frac;
wire [INP_SIZE-1:0] frac2_frac;
wire [INP_SIZE-1:0] frac1_1_append;
wire [INP_SIZE-1:0] frac2_1_append;
wire [INP_SIZE-1:0] frac_extractor1; wire [INP_SIZE-1:0] frac_extractor2;
wire signed [4:0] k_val1_2comp
wire signed [4:0] k_val2_2comp;
wire signed [4+ES:0] total_exp1wire signed [4+ES:0] total_exp2wire signed [5+ES:0]
Diff1_exp;
wire signed [5+ES:0] Diff2_exp;
reg [4+ES:0] Diff_exp_abs
reg [(INP_SIZE-1):0] frac_sum ;
reg [(INP_SIZE-1):0] frac_sum_MSB_intermediate;
```

```
reg [(INP_SIZE-1):0] frac_sum_MSB_removed;
reg signed [4+ES:0] result_exp;
reg signed [4+ES:0] result_exp_norm;
wire [4+ES:0] result_exp_abs;
reg signed [6+ES:0] result_regime;
wire [ES-1:0] result_exp_posit;
reg sign_bit;
reg [3:0] result_k_val;
wire [INP_SIZE-1:0] sum_intermediate;
wire [2:0] result_regime_end_index;
wire [INP_SIZE-1:0] exponent_inserter;
wire [INP_SIZE-1:0] sum_intermediate_exp;
wire [INP_SIZE-1:0] frac_inserter;
wire [INP_SIZE-1:0] sum_unsigned;
reg [INP_SIZE-1:0] shift_test;
reg [3:0] shift_val;
reg [INP_SIZE-1:0] sum_prefinal;
integer i,j;
assign abs_inp1 = (inp1[INP_SIZE-1] == 1'b1) ? (~(inp1) + 1) : inp1;
assign abs_inp2 = (inp2[INP_SIZE-1] == 1'b1) ? (~(inp2) + 1) : inp2;
always@(*) begin
k_val1 = 0;
for (i=0; i<=INP_SIZE-3; i=i+1) begin
if (abs_inp1[i] == 1'b1) begin
k_val1 = INP_SIZE-2-i;
end
end
end
always@(*) begin
k_val2 = 0;
for (i=0; i<=INP_SIZE-3; i=i+1) begin
if (abs_inp2[i] == 1'b1) begin
k_val2 = INP_SIZE-2-i;
end
```

```
end
end
assign regime_end_index1 = INP_SIZE-2-k_val1;
assign regime_end_index2 = INP_SIZE-2-k_val2;
assign exp_extractor1 = {1'b1,{(INP_SIZE-ES-1){1'b0}},{(ES){1'b1}}} <<
(regime_end_index1-ES);
assign exp_extractor2 = {1'b1,{(INP_SIZE-ES-1){1'b0}},{(ES){1'b1}}} <<
(regime_end_index2-ES);
assign exp_intermediate1 = (abs_inp1 & exp_extractor1) >> (regime_end_index1-ES);
assign exp_intermediate2 = (abs_inp2 & exp_extractor2) >> (regime_end_index2-ES);
assign exp1 = exp_intermediate1[ES-1:0];
assign exp2 = exp_intermediate2[ES-1:0];
assign frac_extractor1 = ~({(INP_SIZE-1){1'b1}} << (regime_end_index1-ES));
assign frac_extractor2 = ~({(INP_SIZE-1){1'b1}} << (regime_end_index2-ES));
assign frac1_frac = abs_inp1 & frac_extractor1;
assign frac2_frac = abs_inp2 & frac_extractor2;
assign frac1_1_append = {{(INP_SIZE-2){1'b0}},1'b1} << (regime_end_index1-ES);
assign frac2_1_append = {{(INP_SIZE-2){1'b0}},1'b1} << (regime_end_index2-ES);
assign frac1 = frac1_frac | frac1_1_append;
assign frac2 = frac2_frac | frac2_1_append;
assign frac1_max_shifted_left = frac1 << (k_val1 + 2 + ES -2);
assign frac2_max_shifted_left = frac2 << (k_val2 + 2 + ES -2);
assign k_val1_2comp = ~(k_val1)+1;
assign k_val2_2comp = ~(k_val2)+1;
assign total_exp1 = (k_val1_2comp<<ES) + exp1;
assign total_exp2 = (k_val2_2comp<<ES) + exp2;
assign Diff1_exp = total_exp1 - total_exp2;
assign Diff2_exp = total_exp2 - total_exp1;
always@(*) begin
if(Diff1_exp[5+ES] == 1'b1) begin
Diff_exp_abs = ~(Diff1_exp) + 1;
result_exp = total_exp2;
if (inp1[INP_SIZE-1]^inp2[INP_SIZE-1] == 1'b1) begin
```

```
frac_sum = (frac2_max_shifted_left - (frac1_max_shifted_left >> Diff_exp_abs));
end
else begin
frac_sum = (frac2_max_shifted_left + (frac1_max_shifted_left >> Diff_exp_abs));
end
if (inp2[INP_SIZE-1] == 1'b1) begin
sign_bit = 1'b1;
end
else begin
sign_bit = 1'b0;
end
end
else if (Diff2_exp[5+ES] == 1'b1) begin
Diff_exp_abs = ~(Diff2_exp) + 1;
result_exp = total_exp1;
if (inp1[INP_SIZE-1]^inp2[INP_SIZE-1] == 1'b1) begin
frac_sum = (frac1_max_shifted_left - (frac2_max_shifted_left >> Diff_exp_abs));
end
else begin
frac_sum = (frac1_max_shifted_left + (frac2_max_shifted_left >> Diff_exp_abs));
end
if (inp1[INP_SIZE-1] == 1'b1) begin
sign_bit = 1'b1;
end
else begin
sign_bit = 1'b0;
end
end
else begin
Diff_exp_abs = Diff1_exp;
result_exp = total_exp1;
if(inp1[INP_SIZE-1]^inp2[INP_SIZE-1] == 1'b1)begin
if (frac1_max_shifted_left > frac2_max_shifted_left)begin
frac_sum = frac1_max_shifted_left - frac2_max_shifted_left;
```

```
if (inp1[INP_SIZE-1] == 1'b1) begin
sign_bit = 1'b1;
end
else begin
sign_bit = 1'b0;
end
end
else begin
frac_sum = frac2_max_shifted_left - frac1_max_shifted_left;
if (inp2[INP_SIZE-1] == 1'b1) begin
sign_bit = 1'b1;
end
else begin
sign_bit = 1'b0;
end
end
end
else begin
frac_sum = frac1_max_shifted_left + frac2_max_shifted_left;
if (inp1[INP_SIZE-1] == 1'b1) begin
sign_bit = 1'b1;
end
else begin
sign_bit = 1'b0;
end
end
end
end
always@(*) begin
if(frac_sum[INP_SIZE-1] == 1'b1) begin
frac_sum_MSB_removed = frac_sum << 1'b1;
result_exp_norm = result_exp + 1;
end
else begin
```

```
if(frac_sum[INP_SIZE-2] == 1'b1) begin
frac_sum_MSB_removed = frac_sum << 2'b10;
result_exp_norm = result_exp;
shift_val = 0;
end
else begin
frac_sum_MSB_intermediate = frac_sum << 2'b10;
shift_val = 0;
for (j=0;j<=(INP_SIZE-1);j=j+1) begin
if (frac_sum_MSB_intermediate[j] == 1'b1) begin
shift_val = INP_SIZE - j;
end
end
frac_sum_MSB_removed = frac_sum_MSB_intermediate << (shift_val);
result_exp_norm = result_exp - shift_val;
end
end
end
assign result_exp_abs = ~(result_exp_norm) + 1;
always@(*) begin
if (result_exp_abs < ({{(ES){1'b0}},1'b1} << ES) ) begin
result_k_val = 1 ;
result_regime = 0;
end
else begin
if((result_exp_abs % ({{(ES){1'b0}},1'b1} << ES)) == 0) begin
result_regime = result_exp_abs >> ES;
result_k_val = result_regime[3:0];
end
else begin
result_regime = (result_exp_abs >> ES) + 1;
result_k_val = result_regime[3:0];
end
end
```

```
assign result_exp_posit = result_exp_norm + (({{(ES){1'b0}},1'b1} << ES)*(result_k_val));
assign sum_intermediate = {1'b1,{(INP_SIZE-1){1'b0}}} >> (result_k_val+1);
assign result_regime_end_index = INP_SIZE-1-result_k_val-1;
assign exponent_inserter = {{(INP_SIZE-ES){1'b0}},result_exp_posit} <<
(result_regime_end_index-ES);
assign sum_intermediate_exp = sum_intermediate | exponent_inserter; assign frac_inserter =
frac_sum_MSB_removed >> (result_k_val+1+1+ES);
assign sum_unsigned = sum_intermediate_exp | frac_inserter;
always@(*) begin
if (sign_bit == 1'b1) begin
sum_prefinal = ~(sum_unsigned) + 1;
end
else begin
sum_prefinal = sum_unsigned;
end
end
assign sum = (inp1 == 0) ? inp2 : (inp2 == 0) ? inp1 : ((inp1[INP_SIZE-1]^inp2[INP_SIZE-
1]) & (&(~(abs_inp1^abs_inp2))) == 1) ? {(INP_SIZE){1'b0}} : sum_prefinal ;
endmodule
```

TEST BENCHES FOR ADDER:

```
`timescale 10 us/1 us
`define INP_SIZE 10
module posit_adder_tb;
reg [`INP_SIZE-1:0] inp1,inp2;
wire [`INP_SIZE-1:0] sum ;
posit_adder add1(.inp1(inp1),.inp2(inp2),.sum(sum));
initial begin
inp1 = 10'b1110011001;
inp2 = 10'b0001100101;
#10
inp1 = 10'b0001011011;
inp2 = 10'b1110100101;
```

```
#10
inp1 = 10'b0010000011;
inp2 = 10'b0011100000;
#10
inp1 = 10'b1110000010;
inp2 = 10'b0011000111;
#10
inp1 = 10'b1110000010;
inp2 = 10'b0011000111;
#10
inp1 = 10'b0001110010;
inp2 = 10'b0011011000;
#10
inp1 = 10'b1110011001;
inp2 = 10'b0001100101;
#10
inp1 = 10'b0000011110;
inp2 = 10'b0010101011;
#10
inp1 = 10'b1111000011;
inp2 = 10'b0001100100;
#10
inp1 = 10'b0011010110;
inp2 = 10'b0001011101;
#10
$stop;
end
endmodule
```

VERILOG CODE FOR MULTIPLIER:

```
module posit_multiplier #(parameter INP_SIZE = 10,
parameter ES = 2)
(
input [INP_SIZE-1:0] inp1,inp2,
output [INP_SIZE-1:0] product;
wire [INP_SIZE-1:0] abs_inp1, abs_inp2;
reg [3:0] k_val1, k_val2;  wire [2:0] regime_end_index1;
wire [2:0] regime_end_index2; wire [ES-1:0] exp1;
wire [ES-1:0] exp2;
wire [INP_SIZE-1:0] exp_extractor1wire [INP_SIZE-1:0] exp_extractor2;
wire [INP_SIZE-1:0] exp_intermediate1; wire [INP_SIZE-1:0] exp_intermediate2;
wire [INP_SIZE-1:0] frac1;
wire [INP_SIZE-1:0] frac2;
wire [INP_SIZE-1:0] frac1_frac;
wire [INP_SIZE-1:0] frac2_frac;
wire [INP_SIZE-1:0] frac1_1_append;
wire [INP_SIZE-1:0] frac2_1_append;
wire [INP_SIZE-1:0] frac_extractor1; wire [INP_SIZE-1:0] frac_extractor2;
wire signed [4:0] k_val1_2comp;
wire signed [4:0] k_val2_2comp;
wire signed [4+ES:0] total_exp1wire signed [4+ES:0] total_exp2)
wire signed [5+ES:0] result_exp;
wire [2*(INP_SIZE-3-ES)+1:0] result_frac; reg  signed [6+ES:0] result_exp_norm;
wire [6+ES:0] result_exp_abs;
wire [3:0] result_frac_MSB_index;
reg [3:0] result_frac_MSB_index_final;//
reg signed [6+ES:0] result_regime;
wire [ES-1:0] result_exp_posit; // same as input exp bits
wire sign_bit;
reg [3:0] result_k_val;
wire [INP_SIZE-1:0] product_intermediate;
wire [2:0] result_regime_end_index;
```

```
wire [INP_SIZE-1:0] exponent_inserter;
wire [INP_SIZE-1:0] product_intermediate_exp;
wire [INP_SIZE-1:0] frac_inserter;
wire [INP_SIZE-1:0] product_unsigned;
wire [2*(INP_SIZE-3-ES)+1:0] frac_MSB_removed;
reg [INP_SIZE-1:0] product_prefinal;
integer i;
assign abs_inp1 = (inp1[INP_SIZE-1] == 1'b1) ? (~(inp1) + 1) : inp1;
assign abs_inp2 = (inp2[INP_SIZE-1] == 1'b1) ? (~(inp2) + 1) : inp2;
always@(*) begin
k_val1 = 0;
for (i=0; i<=INP_SIZE-3; i=i+1) begin
if (abs_inp1[i] == 1'b1) begin
k_val1 = INP_SIZE-2-i;
end
end
end
always@(*) begin
k_val2 = 0;
for (i=0; i<=INP_SIZE-3; i=i+1) begin
if (abs_inp2[i] == 1'b1) begin
k_val2 = INP_SIZE-2-i;
end
end
end
assign regime_end_index1 = INP_SIZE-2-k_val1;
assign regime_end_index2 = INP_SIZE-2-k_val2;
assign exp_extractor1 = {1'b1,{(INP_SIZE-ES-1){1'b0}},{(ES){1'b1}}} <<
(regime_end_index1-ES);
assign exp_extractor2 = {1'b1,{(INP_SIZE-ES-1){1'b0}},{(ES){1'b1}}} <<
(regime_end_index2-ES);
assign exp_intermediate1 = (abs_inp1 & exp_extractor1) >> (regime_end_index1-ES);
assign exp_intermediate2 = (abs_inp2 & exp_extractor2) >> (regime_end_index2-ES);
assign exp1 = exp_intermediate1[ES-1:0];
```

```
assign exp2 = exp_intermediate2[ES-1:0];
assign frac_extractor1 = ~({(INP_SIZE-1){1'b1}} << (regime_end_index1-ES));
assign frac_extractor2 = ~({(INP_SIZE-1){1'b1}} << (regime_end_index2-ES));
assign frac1_frac = abs_inp1 & frac_extractor1;
assign frac2_frac = abs_inp2 & frac_extractor2;
assign frac1_1_append = {{(INP_SIZE-2){1'b0}},1'b1} << (regime_end_index1-ES);
assign frac2_1_append = {{(INP_SIZE-2){1'b0}},1'b1} << (regime_end_index2-ES);
assign frac1 = frac1_frac | frac1_1_append;
assign frac2 = frac2_frac | frac2_1_append;
prepare
assign k_val1_2comp = ~(k_val1)+1;
assign k_val2_2comp = ~(k_val2)+1;
assign total_exp1 = (k_val1_2comp<<ES) + exp1;
assign total_exp2 = (k_val2_2comp<<ES) + exp2;
assign result_exp = total_exp1 + total_exp2;
assign result_frac = frac1 * frac2;
assign result_frac_MSB_index = 2 + regime_end_index1-2 + regime_end_index2-2 - 1 ;
always@(*) begin
if (result_frac[result_frac_MSB_index]==1) begin
result_exp_norm = result_exp + 1;
result_frac_MSB_index_final = result_frac_MSB_index-1;
end
else begin
result_exp_norm = result_exp;
result_frac_MSB_index_final = result_frac_MSB_index-2;
end
end
assign result_exp_abs = ~(result_exp_norm) + 1;
always@(*) begin
if (result_exp_abs < ({{(ES){1'b0}},1'b1} << ES) ) begin
result_k_val = 1 ;
result_regime = 0;
end
else begin
```

```
if((result_exp_abs % ({{(ES){1'b0}},1'b1} << ES)) == 0) begin
result_regime = result_exp_abs >> ES;
result_k_val = result_regime[3:0];
end
else begin
result_regime = (result_exp_abs >> ES) + 1;
result_k_val = result_regime[3:0];
end
end
end
assign result_exp_posit = result_exp_norm + (({{(ES){1'b0}},1'b1} << ES)*(result_k_val));
assign product_intermediate = {1'b1,{(INP_SIZE-1){1'b0}}} >> (result_k_val+1);
assign result_regime_end_index = INP_SIZE-1-result_k_val-1;
assign exponent_inserter = {{(INP_SIZE-ES){1'b0}},result_exp_posit} <<
(result_regime_end_index-ES);
assign product_intermediate_exp = product_intermediate | exponent_inserter; // product with
exp value encoded
assign frac_MSB_removed = result_frac << ((2*(INP_SIZE-3-ES)+1) -
(result_frac_MSB_index_final));
assign frac_inserter = frac_MSB_removed >> (((2*(INP_SIZE-3-ES)+1)-(INP_SIZE-1)) +
(result_k_val+1+1+ES));
assign product_unsigned = product_intermediate_exp | frac_inserter;
always@(*) begin
if (inp1[INP_SIZE-1]^inp2[INP_SIZE-1]==1) begin
product_prefinal = ~(product_unsigned) + 1;
end
else begin
product_prefinal = product_unsigned;
end
end
assign product = ((inp1 & inp2) == 0) ? {(INP_SIZE){1'b0}} : product_prefinal;
endmodule
```

TEST BENCHES FOR MULTIPLER:

```
`timescale 10 us/1 us
`define INP_SIZE 10
module posit_multiplier_tb;
reg [`INP_SIZE-1:0] inp1,inp2;
wire [`INP_SIZE-1:0] product;
posit_multiplier mult1(.inp1(inp1),.inp2(inp2),.product(product));
initial begin
inp1 = 10'b1110011001;
inp2 = 10'b0001100101;
#10
inp1 = 10'b1110011001;
inp2 = 10'b0000000000;
#10
inp1 = 10'b0011101101;
inp2 = 10'b1111111001;
#10
inp1 = 10'b0001110010;
inp2 = 10'b0011011000;
#10
inp1 = 10'b1110000010;
inp2 = 10'b0011000111;
#10
inp1 = 10'b0000011110;
inp2 = 10'b0010101011;
#10
inp1 = 10'b1111000011;
inp2 = 10'b0001100100;
#10
$stop;
end
endmodule
```

VERILOG CODE FOR FILTER

```
module fir #(parameter INP_SIZE = 10,
parameter COEFF_SIZE = 10,
parameter ORDER = 32)
(input [INP_SIZE-1 : 0] fir_in,
input fir_clk,
input fir_rst,
output [INP_SIZE - 1 : 0] fir_out);
reg [INP_SIZE-1 : 0] inp_reg;
reg [INP_SIZE + COEFF_SIZE - 1 : 0] outp_reg;
wire input [0 : COEFF_SIZE*((ORDER/2)+1) - 1] fir_coeff_inp;
reg [COEFF_SIZE-1 : 0] fir_coeff [0 : (ORDER/2)];
wire [(INP_SIZE) - 1 : 0] mult_out [0 : ORDER/2];
wire [(INP_SIZE) - 1 : 0] reg_out [0 : ORDER - 1];
wire [(INP_SIZE) - 1 : 0] add_out [0 : ORDER - 1];
integer j;
always@(posedge fir_clk) begin
if(fir_rst == 1'b1) begin
inp_reg <= 0;
end
else begin
inp_reg <= fir_in;
end
end
assign fir_coeff_inp =
{10'd975,10'd961,10'd944,10'd932,10'd923,10'd916,10'd912,10'd911,10'd915,10'd927,10'd56
,10'd107,10'd123,10'd137,10'd149,10'd157,10'd159};
always@(*) begin
for (j=0; j<= ORDER/2; j=j+1) begin
fir_coeff[j] <= fir_coeff_inp[(j*COEFF_SIZE) +: COEFF_SIZE];
end
end
genvar i;
```

```
generate
for (i = 0; i<= ORDER/2; i = i + 1) begin : gen0
#(.MULT_INP1_SIZE(INP_SIZE), .MULT_INP2_SIZE(COEFF_SIZE)) m0
multiplier m1(.a(fir_in), .b(fir_coeff[i]), .p(mult_out[i]));
end
endgenerate
generate
for (i = 0; i<= (ORDER/2) - 1; i = i + 1) begin : gen1
#(.ADD_INP1_SIZE(INP_SIZE + COEFF_SIZE), .ADD_INP2_SIZE(INP_SIZE +
COEFF_SIZE)) a0
adder a1(.a(reg_out[i]), .b(mult_out[i+1]), .outp(add_out[i]));
end
endgenerate
generate
for (i = ORDER/2; i <= (ORDER - 1); i= i + 1) begin : gen2
#(.ADD_INP1_SIZE(INP_SIZE + COEFF_SIZE), .ADD_INP2_SIZE(INP_SIZE +
COEFF_SIZE)) a1
adder a2(.a(reg_out[i]), .b(mult_out[ORDER-1-i]), .outp(add_out[i]));
end
endgenerate
intermediate_regs #(.REG_INP_SIZE(INP_SIZE + COEFF_SIZE))
ir0 (.reg_in(mult_out[0]), .clk(fir_clk), .rst(fir_rst),
.reg_out(reg_out[0]));
generate
for (i = 1; i <= (ORDER - 1); i = i + 1) begin : gen3
intermediate_regs #(.REG_INP_SIZE(INP_SIZE + COEFF_SIZE))
ir0 (.reg_in(add_out[i-1]), .clk(fir_clk),
.rst(fir_rst),
.reg_out(reg_out[i]));
end
endgenerate
always@(posedge fir_clk) begin
if(fir_rst == 1'b1) begin
outp_reg <= 0;
```

```
end
else begin
outp_reg <= add_out[ORDER-1];
end
end
assign fir_out = outp_reg;
endmodule
```

TEST BENCH FOR FILTER:

```
`timescale 1 ms/10 us
`define INP_SIZE 10
`define COEFF_SIZE  10
`define ORDER  24
`define CHANNELS  8
`define BANDS  5
`define SAMPLES 94
module fir_tb;
reg [`INP_SIZE-1 : 0] fir_in;
reg clk; reg rst;
wire [(`INP_SIZE + `COEFF_SIZE) -1 : 0] fir_out ;
localparam clk_period = 8;
reg [INP_SIZE-1:0] fd1 [0:`SAMPLES-1];
reg [INP_SIZE-1:0] fd2 [0:`SAMPLES-1];
integer i;
fir #(10,10,32) fir_sys0 (system_inp,clk,rst,system_outp);
always
begin
clk = 1'b1;
#(clk_period/2);
clk = 1'b0;
#(clk_period/2);
end
initial
```

```
begin
$readmemb("test_inputs.txt", fd1);
rst = 1'b1;   //reset to initialise the registers
#(clk_period*3);
#(clk_period/2);
rst = 1'b0;
for (i = 0; i < `SAMPLES; i = i + 1)
begin
$display("%b:%d", fd1[i], i);
system_inp <= fd1[i];
#(clk_period);
fd2[i] <= fir_out; end
$writememb("hw_outputs.txt", fd2);  txt file
#(clk_period);
$stop;
end
endmodule
```

MATLAB CODE FOR COEFFICIENT GENERATION

```
clear all;
close all;
clc;
Fs = 256; %sampling frequency
N =  33 ;%taps
no_of_bits = 9;
f1 = 8;%7.4;%7.4;  %7.59
f2 = 13;%13.61;%13.61; %13.42
f = [f1 f2]/(Fs/2); %normalised frequency
hc_unquantised = fir1(N-1, f,'bandpass',kaiser(N,5.65)); % coefficients
no_of_bits = 9;
x_quant_floor = floor(hc_unquantised*2^(no_of_bits))/2^(no_of_bits);
x_quant_ceil = ceil(hc_unquantised*2^(no_of_bits))/2^(no_of_bits);
x_quant = zeros(1,N);
```

```
for i = 1:N
if (hc_unquantised(i) - x_quant_floor(i)) >= (x_quant_ceil(i) - hc_unquantised(i))
x_quant(i) = x_quant_ceil(i);
else
x_quant(i) = x_quant_floor(i);
end
end
for i = 1:N
if(x_quant(i) == -0)
x_quant(i) = 0;
end
end
x_quant1 = fi(hc_unquantised,1,10,9);
error = x_quant1 - x_quant
fft_points=4096;  %4096 point fft
fft_val= 20*log10(abs(fftshift(fft(hc_unquantised,fft_points)))); %fft from -fs/2 to fs/2
freq_x_axis=(-0.5:1/fft_points:0.5-1/fft_points)*Fs; %to display freq on x-axis
figure
plot(freq_x_axis,fft_val)
title('Gamma - Filter Frequency Response (Unquantised coefficients)')
xlabel("Frequency")
ylabel("Magnitude (dB)")
grid on
hold on
three_db_line = (-3) * ones(1, length(freq_x_axis)); %-3dB line
plot(freq_x_axis,three_db_line)
fft_points=4096;  %4096 point fft
fft_val= 20*log10(abs(fftshift(fft(x_quant,fft_points)))); %fft from -fs/2 to fs/2
freq_x_axis=(-0.5:1/fft_points:0.5-1/fft_points)*Fs; %to display freq on x-axis
figure
plot(freq_x_axis,fft_val)
title('alpha- Filter Frequency Response (10-Bit Quantized coefficients)')
xlabel("Frequency")
ylabel("Magnitude (dB)")
```

```
grid on
hold on
three_db_line = (-3) * ones(1, length(freq_x_axis)); %-3dB line
plot(freq_x_axis,three_db_line)
hc_posit = readmatrix('Coefficients_Final\decimal_equivalent_of_posit\alpha_p_to_d.txt')
fft_points=4096;  %4096 point fft
fft_val= 20*log10(abs(fftshift(fft(hc_posit,fft_points))));
freq_x_axis=(-0.5:1/fft_points:0.5-1/fft_points)*Fs;
figure
plot(freq_x_axis,fft_val)
title('alpha- Filter Frequency Response (10-bit Posit coefficients)')
xlabel("Frequency")
ylabel("Magnitude (dB)")
grid on
hold on
three_db_line = (-3) * ones(1, length(freq_x_axis)); %-3dB line
plot(freq_x_axis,three_db_line)
```

MATLAB CODE FOR FILTER TEST INPUT GENERATION:

```
clear all;
close all;
Fs = 256;
dt = 1/Fs;
fin1 = 40;
fin2 = 15;
samples = 800;
cycles = fin1*samples/Fs;
%cycles = 15;
StopTime = (1/fin1)*cycles;
t = (0:dt:StopTime);
%%Sine wave:
x = 0.5 * sin(2*pi*fin1*t)  %+ 0.5 * sin(2*pi*(fin2)*t);
total_samples = length(x)
```

```
no_of_bits = 14;
x_quant_floor = floor(x*2^(no_of_bits))/2^(no_of_bits);
x_quant_ceil = ceil(x*2^(no_of_bits))/2^(no_of_bits);
x_quant = zeros(1,total_samples);
for i = 1:total_samples
if (x(i) - x_quant_floor(i)) >= (x_quant_ceil(i) - x(i))
x_quant(i) = x_quant_ceil(i);
else
x_quant(i) = x_quant_floor(i);
end
end
for i = 1:total_samples
if(x_quant(i) == -0)
x_quant(i) = 0;
end
end
x_quant1 = fi(x,1,15,14);
error = x_quant1 - x_quant
x_filter_hdl = zeros(1,total_samples);
for i = 1:total_samples
x_filter_hdl(i) = x_quant1(i)*2^(no_of_bits);
if (x_quant(i)<0)
x_filter_hdl(i) = 2^(no_of_bits+1) + x_filter_hdl(i);
end
end
figure;
stem(t,x);
xlabel('time (in sec)');
ylabel("Amplitude")
title('Input Signal versus Time');
figure;
stem(t,x_quant1);
xlabel('time (in sec)');
ylabel("Amplitude")
```

```
title('Quantized Input Signal versus Time');
fft_points=4096;  %4096 point fft
fft_val= 20*log10(abs(fftshift(fft(x_quant,fft_points)))); %fft from -fs/2 to fs/2
freq_x_axis=(-0.5:1/fft_points:0.5-1/fft_points)*Fs; %to display freq on x-axis
figure
stem(freq_x_axis,fft_val)
x_quant1
format long
writematrix(x_quant(:),"Inputs_final\raw_inputs\gamma_ip_raw.txt");
```

PYTHON CODES FOR POSIT/BINARY/DECIMAL CONVERSION:

BINARY- POSIT (PYTHON CODE):

```
import math
def bin_to_posit(binary_str):
comp=""
check=comp.zfill((len(binary_str)))
print("length of str",len(binary_str))
print("check", check)
if binary_str==check:
print("inside if")
return binary_str
else:
print("inside else")
def twos_complement(binary_string):
if not binary_string:
raise ValueError("Input string is empty.")
if binary_string[0] == '1':
inverted = ''.join('1' if bit == '0' else '0' for bit in binary_string)
twos_comp = bin(int(inverted, 2) + 1)[2:]
twos_comp = twos_comp.zfill(len(binary_string))
return twos_comp
else:
```

```
print(twos_complement(binary_str))
intvalue= int(twos_complement(binary_str),base=2)
intvalue=intvalue/(2**(len(binary_str)-1))
print("intvalue ",intvalue)
es=2
useed= 2**(2**es)
k=0
def adjust_to_useed_range(num,k):
if num == 0:
return num
else :
while num < 1 or num >= useed:
if num < 1:
num *= useed  # Multiply by useed if less than 1
k -= 1
elif num >= useed:
num /= useed  # Divide by 16 if greater than or equal to useed
k += 1
return num,k
'''try:
number = intvalue
result = adjust_to_range(number)
print(f"The number adjusted to the range [1, 16) is: {result}")
except ValueError as e:
print(f"Error: {e}")'''
uscale,k=adjust_to_useed_range(intvalue,k)
print("uscale ",uscale, "k value ",k)
exp=0
def adjust_to_two_range(num,exp):
while not (1 <= num < 2):
num /= 2  # Divide by 2 only because the useed scaling factor has made the value >1
exp+=1
return num,exp
number = float(input("Enter a number: "))
```

```
adjusted_number = adjust_to_range(number)
print(f"The adjusted number within range [1, 16) is: {adjusted_number}")
except ValueError as e:
print(f"Error: {e}")'''
normalized_value,exp=adjust_to_two_range(uscale,exp)
print("fraction value ",normalized_value," and exponent value",exp)
def separate_decimal_fraction(num):
fractional_part, integer_part = math.modf(num)
return int(integer_part), fractional_part
integer,fraction=separate_decimal_fraction(normalized_value)
print("fraction part", fraction, "and integer part", integer )
mantissa_decimal= int(fraction*(2**(len((binary_str))-1)))
print("mantissa in decimal ",mantissa_decimal)
mantissa_binary=bin(mantissa_decimal)[2:]
mantissa_binary=mantissa_binary.zfill((len(binary_str)-1))
print("mantisssa in binary ", mantissa_binary)
exp_bin = format(exp, f'0{es}b')
print("exp_bin value",exp_bin)
def process_input(value):
if value >= 0:
result = "1" * (value + 1) + "0"
else:
result = "0" * abs(value) + "1"
return result
try:
input_value = int(input("Enter an integer: "))
output = process_input(input_value)
print(f"Output: {output}")
except ValueError:
print("Invalid input. Please enter an integer.")'''
regime_bits= process_input(k)
concat_op= "0"+ regime_bits + exp_bin + mantissa_binary
truncated_op= concat_op[:(len(binary_str))]
ip_signbit= binary_str[0]
```

```
if ip_signbit==1 :'"
def p_twos_complement(binary_string,ip_signbit):
if not binary_string:
raise ValueError("Input string is empty.")
if ip_signbit == '1':
inverted = ''.join('1' if bit == '0' else '0' for bit in binary_string)
twos_comp = bin(int(inverted, 2) + 1)[2:] #removes the '0b' prefix
twos_comp = twos_comp.zfill(len(binary_string))
return twos_comp
else:
posit_no= p_twos_complement(truncated_op,ip_signbit)
print("binary no ",binary_str)
print("posit number ",posit_no)
return posit_no
input_file_path= "C:\\babu\\college related\\7th sem\\Project\\Codes\\test_inputs.txt"
output_file_path= "C:\\babu\\college related\\7th
sem\\Project\\Codes\\test_inputs_posit_converted.txt"
with open(input_file_path, 'r') as input_file, open(output_file_path, 'w') as output_file:
for line in input_file:
print(line)
processed_line = bin_to_posit(line.strip())
output_file.write(processed_line + "\n")
print("Processing complete. Check the output file.")
```

DECIMAL – POSIT (PYTHON CODE):

```
import math
def decimal_to_posit(float_val):
nbits=10
if float_val==0:
posit_no=bin((int(float_val)))[2:]
posit_no_nbits= posit_no.zfill(nbits)
return posit_no_nbits
else:
```

```
s=2
useed= 2**(2**es)
k=0
def adjust_to_useed_range(num,k):
if num == 0:
return num
else :
while num < 1 or num >= useed:
if num < 1:
num *= useed
k -= 1
elif num >= useed:
num /= useed
k += 1
return num,k
'''try:
number = intvalue
result = adjust_to_range(number)
print(f"The number adjusted to the range [1, 16) is: {result}")
except ValueError as e:
print(f"Error: {e}")'''
uscale,k=adjust_to_useed_range(abs(float_val),k)
print("uscale ",uscale, "k value ",k)
exp=0
def adjust_to_two_range(num,exp):
while not (1 <= num < 2):
num /= 2
exp+=1
return num,exp
'''try:
number = float(input("Enter a number: "))
adjusted_number = adjust_to_range(number)
print(f"The adjusted number within range [1, 16) is: {adjusted_number}")
except ValueError as e:
```

```
print(f"Error: {e}")'''
normalized_value,exp=adjust_to_two_range(uscale,exp)
print("fraction value ",normalized_value," and exponent value",exp)
def separate_decimal_fraction(num):
fractional_part, integer_part = math.modf(num)
return int(integer_part), fractional_part
integer,fraction=separate_decimal_fraction(normalized_value)
print("fraction part", fraction, "and integer part", integer )
mantissa_decimal= int(fraction*(2**(nbits-1)))
print("mantissa in decimal ",mantissa_decimal)
mantissa_binary=bin(mantissa_decimal)[2:]
mantissa_binary=mantissa_binary.zfill((nbits-1))
print("mantisssa in binary ", mantissa_binary)
exp_bin = format(exp, f'0{es}b')
print("exp_bin value",exp_bin)
def process_input(value):
if value >= 0:
result = "1" * (value + 1) + "0"
else:
result = "0" * abs(value) + "1"
return result
try:
input_value = int(input("Enter an integer: "))
output = process_input(input_value)
print(f"Output: {output}")
except ValueError:
print("Invalid input. Please enter an integer.")'''
regime_bits= process_input(k)
concat_op= "0"+ regime_bits + exp_bin + mantissa_binary
truncated_op= concat_op[:nbits]
'''def p_twos_comp()
if ip_signbit==1 :'''
def p_twos_complement(binary_string,flo_val):
if not binary_string:
```

```
raise ValueError("Input string is empty.")
if flo_va<0:
inverted = ''.join('1' if bit == '0' else '0' for bit in binary_string)
twos_comp = bin(int(inverted, 2) + 1)[2:] #removes the '0b' prefix
twos_comp = twos_comp.zfill(len(binary_string))
return twos_comp
else:
posit_no= p_twos_complement(truncated_op,float_val)
print("decimal no",float_val)
print("posit number ",posit_no)
return posit_no
input_file_path= "C:\\babu\\college related\\7th
sem\\Project\\Codes\\Minimum_product_value_decimal.txt"
output_file_path= "C:\\babu\\college related\\7th
sem\\Project\\Codes\\Minimum_product_value_posit_converted.txt"
with open(input_file_path, 'r') as input_file, open(output_file_path, 'w') as output_file:
for line in input_file:
print(line)
processed_line = decimal_to_posit(float(line))
output_file.write(processed_line + "\n")
print("Processing complete. Check the output file.")
```

POSIT – BINARY (PYTHON CODE):

```
def posit_to_binary(posit_str):
comp=""
check=comp.zfill((len(posit_str)))
print("length of str",len(posit_str))
print("check", check)
if posit_str==check:
print("inside if")
return posit_str
else:
print("inside else")
```

```
def twos_complement(posit_string):
if not posit_string:
raise ValueError("Input string is empty.")
if posit_string[0] == '1':
inverted = ''.join('1' if bit == '0' else '0' for bit in posit_string)
twos_comp = bin(int(inverted, 2) + 1)[2:] #removes the '0b' prefix
twos_comp = twos_comp.zfill(len(posit_str))
return twos_comp
else:
return posit_string
two_comp= twos_complement(posit_str)
print("2's comp ", two_comp)
def k_value_from_regimebits(input_string):
k = 0
index_at_break = 0
if not input_string:
raise ValueError("Input string cannot be empty.")
starting_index=0
initial_char = input_string[starting_index]
if initial_char == '0':
print("inside 0 for")
m = 0
for i, char in enumerate(input_string):
print("i value",i)
print("char value",char)
if char == '0':
m += 1
elif char == '1':
k = -m
index_at_break = i+1
print("break at ",index_at_break)
break
elif initial_char == '1':
print("inside 1 for")
```

```
m = 0
for i, char in enumerate(input_string):
print("i value",i)
print("char value",char)
if char == '1':
m += 1
elif char == '0':
k = m - 1
index_at_break = i+1
break
if index_at_break == 0:
raise ValueError("No break condition found in the string.")
return k, index_at_break
k,indexbreak=k_value_from_regimebits(two_comp[1:])
print("k value", k)
print("index break value", indexbreak)
es=2
exp_bin=two_comp[(indexbreak+1):(indexbreak+es+1)]
print("exp binary value ", exp_bin)
exp=int((exp_bin),base=2)
print("exp value ",exp)
useed=2**(2**es)
_part_bin=two_comp[(indexbreak+es+1):]
print("fraction part", f_part_bin)
f_part_intermediate= int(f_part_bin,base=2)
f_part=f_part_intermediate/(2**(len(f_part_bin)))
print("fraction part in decimal", f_part)
f_part_posit=1+f_part
print("f_part_posit ",f_part_posit)
print("useed ",useed)
posit_deci=((-1)**(int(posit_str[0],base=2)))*(useed**(k))*(2**(exp))*f_part_posit
print("posit_decimal ",posit_deci)
return posit_deci
```

```
input_file_path= "C:\\babu\\college related\\7th
sem\\Project\\Codes\\coeffs_gamma_unquantised_posit_converted.txt"
output_file_path= "C:\\babu\\college related\\7th
sem\\Project\\Codes\\coeffs_gamma_unquantised_posit_converted_back_to_decimal.txt"
with open(input_file_path, 'r') as input_file, open(output_file_path, 'w') as output_file:
for line in input_file:
print(line)
processed_line = posit_to_binary(line.strip())
output_file.write(f"{processed_line}\n" )
print("Processing complete. Check the output file.")
```